

PROTEÇÃO DE APIs E AUTENTICAÇÃO JWT

Desenvolvimento Web Back-End

Segurança, Autenticação e Autorização

Professor: Geovani

Instituição: Faculdade Einstein de Limeira

Curso: TADS - Análise e Desenvolvimento de Sistemas

Ano: 2026

PROTEÇÃO DE APIs E AUTENTICAÇÃO JWT

Disciplina: Desenvolvimento Web Back-End

Professor: Geovani

Instituição: Faculdade Einstein de Limeira

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas (TADS)

Ano: 2026

SUMÁRIO

1. [Introdução à Segurança em APIs](#)
 2. [Autenticação vs Autorização](#)
 3. [Principais Ameaças em APIs](#)
 4. [Métodos de Autenticação](#)
 5. [JWT - JSON Web Token](#)
 6. [Implementação JWT em ASP.NET Core](#)
 7. [Boas Práticas de Segurança](#)
 8. [Refresh Tokens](#)
 9. [HTTPS e Outras Camadas de Segurança](#)
 10. [Checklist de Segurança](#)
-

1. INTRODUÇÃO À SEGURANÇA EM APIs

1.1 Por que Proteger APIs?

APIs REST são **pontos de entrada** para sua aplicação. Sem proteção adequada:

- ✗ **Dados sensíveis** podem ser expostos
- ✗ **Operações não autorizadas** podem ser executadas
- ✗ **Recursos** podem ser abusados (DoS, spam)
- ✗ **Identidade de usuários** pode ser roubada
- ✗ **Dados** podem ser manipulados ou deletados

1.2 Cenário Real

Sem Proteção:

```
GET https://api.empresa.com/api/usuarios
// ❌ Qualquer um pode acessar lista de usuários!

DELETE https://api.empresa.com/api/usuarios/123
// ❌ Qualquer um pode deletar usuários!
```

Com Proteção:

```
GET https://api.empresa.com/api/usuarios
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
// ✅ Apenas usuários autenticados e autorizados podem acessar!
```

1.3 Camadas de Segurança

1. HTTPS (Criptografia Transporte)
2. Autenticação (Quem é você?)
3. Autorização (O que pode fazer?)
4. Validação de Entrada
5. Rate Limiting
6. Logging e Monitoramento

2. AUTENTICAÇÃO VS AUTORIZAÇÃO

2.1 Conceitos Fundamentais

Autenticação (Authentication)

"QUEM é você?"

Processo de **verificar a identidade** do usuário.

Exemplo: - Login com email e senha - Login com Google/Facebook - Biometria (digital, face)

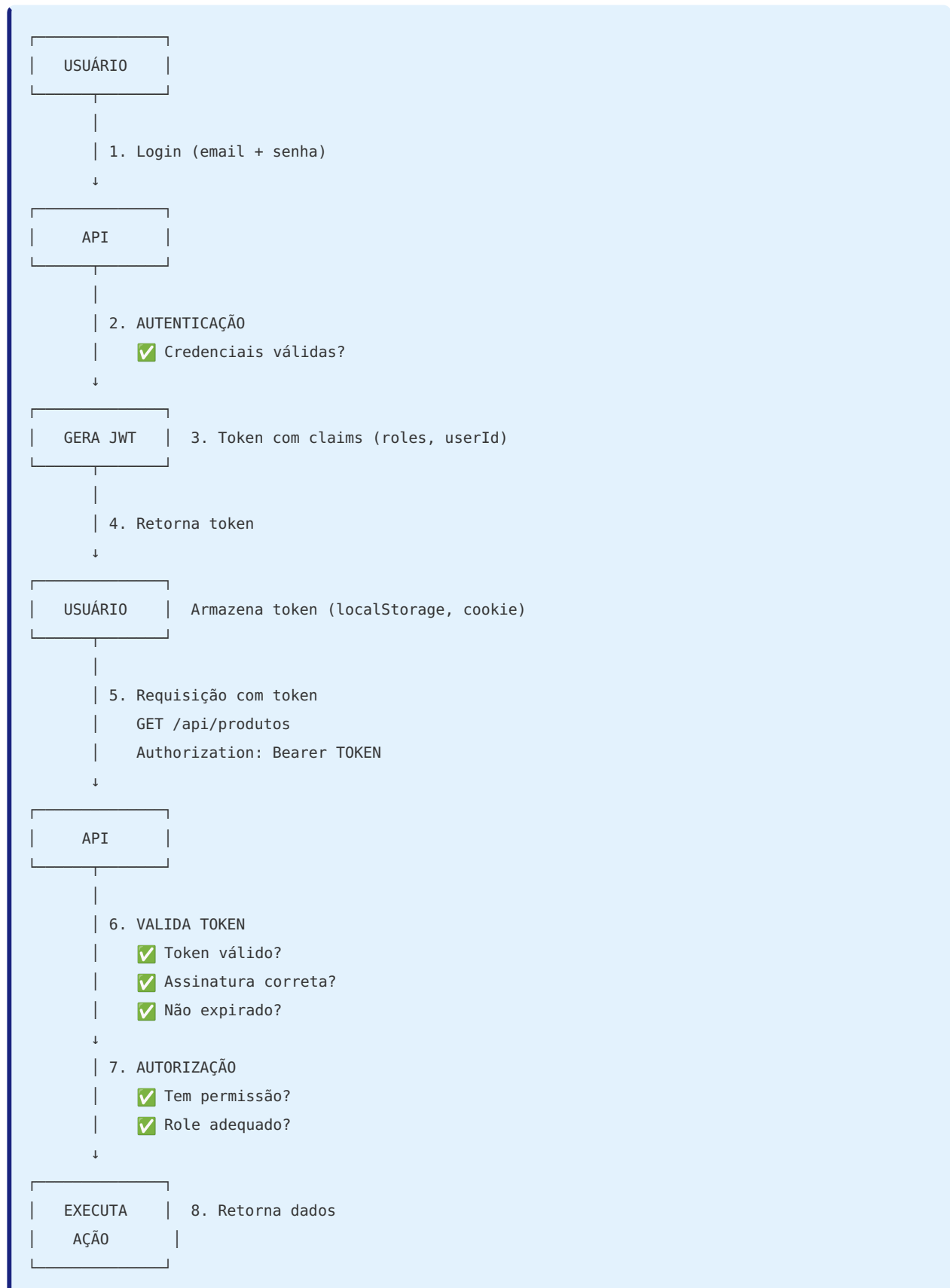
Autorização (Authorization)

"O QUE você pode fazer?"

Processo de **verificar as permissões** do usuário.

Exemplo: - Usuário comum: pode ver dados - Administrador: pode ver, criar, editar, deletar

2.2 Fluxo Completo



2.3 Tabela Comparativa

Aspecto	Autenticação	Autorização
Pergunta	Quem é você?	O que pode fazer?
Processo	Verifica identidade	Verifica permissões
Quando	No login	Em cada requisição
Resultado	Token/Sessão	Acesso permitido/negado
Exemplo	Login com senha	Acesso admin-only
Status Code Erro	401 Unauthorized	403 Forbidden

3. PRINCIPAIS AMEAÇAS EM APIs

3.1 OWASP API Security Top 10

1. Broken Object Level Authorization (BOLA)

Problema: Usuário acessa recursos de outros usuários.

```
❌ MAU:
GET /api/usuarios/123/pedidos
// Usuário 456 pode ver pedidos do usuário 123!

✅ BOM:
// Verificar se usuário logado = usuário do recurso
if (userId != loggedUserId)
    return Forbidden();
```

2. Broken Authentication

Problema: Falhas no processo de autenticação.

- ❌ Senhas fracas permitidas
- ❌ Tokens sem expiração
- ❌ Sem rate limiting em login
- ❌ Credenciais em URLs

3. Excessive Data Exposure

Problema: API retorna mais dados que o necessário.

❌ MAU:

```
return Ok(usuario); // Retorna TUDO (senha, dados internos)
```

✅ BOM:

```
return Ok(new UsuarioDTO
{
    Id = usuario.Id,
    Nome = usuario.Nome,
    Email = usuario.Email
    // Sem senha, dados sensíveis
});
```

4. Lack of Resources & Rate Limiting

Problema: API pode ser abusada (DoS, brute force).

✅ **Solução:** Limitar requisições por IP/usuário.

5. Broken Function Level Authorization

Problema: Endpoints admin acessíveis por usuários comuns.

❌ MAU:

```
[HttpDelete("{id}")]
public IActionResult Delete(int id) { } // Qualquer um pode deletar!
```

✅ BOM:

```
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult Delete(int id) { } // Apenas admin
```

3.2 Outras Ameaças

- **SQL Injection** - Injeção de código SQL
 - **XSS** - Cross-Site Scripting
 - **CSRF** - Cross-Site Request Forgery
 - **Man-in-the-Middle** - Interceptação de dados (sem HTTPS)
 - **Token Hijacking** - Roubo de tokens
-

4. MÉTODOS DE AUTENTICAÇÃO

4.1 Comparação de Métodos

Método	Como Funciona	Vantagens	Desvantagens
Basic Auth	Base64(user:pass) no header	Simples	Inseguro, credenciais em cada request
Session/ Cookie	Servidor armazena sessão	Tradicional	Não escalável, stateful
OAuth 2.0	Delegação de acesso (Google, FB)	Seguro, sem expor senha	Complexo
JWT	Token autocontido assinado	Stateless, escalável	Token maior, revogação difícil
API Keys	Chave única por cliente	Simples para APIs públicas	Menos seguro para usuários

4.2 Por que JWT?

- ✓ **Stateless** - servidor não precisa armazenar sessões
- ✓ **Escalável** - funciona em múltiplos servidores
- ✓ **Cross-domain** - funciona em diferentes domínios
- ✓ **Mobile-friendly** - ideal para apps mobile
- ✓ **Autocontido** - contém informações do usuário
- ✓ **Padrão da indústria** - amplamente adotado

5. JWT - JSON WEB TOKEN

5.1 O que é JWT?

JWT = **J**SON **W**eb **T**oken

Um **token compacto e autocontido** para transmitir informações de forma segura entre partes como um objeto JSON assinado digitalmente.

5.2 Estrutura do JWT

Um JWT é composto por **3 partes** separadas por **ponto (.)**:

```
HEADER . PAYLOAD . SIGNATURE
```

Exemplo Real:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflK
```

Decodificado:

1. HEADER (Cabeçalho)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **alg** : Algoritmo de assinatura (HMAC SHA256, RSA)
- **typ** : Tipo do token (JWT)

2. PAYLOAD (Carga Útil)

```
{
  "sub": "1234567890",
  "name": "João Silva",
  "email": "joao@email.com",
  "role": "Admin",
  "exp": 1735689600
}
```

- **Claims** - informações sobre o usuário
- **sub** - Subject (ID do usuário)
- **name, email, role** - dados customizados
- **exp** - Expiração (timestamp Unix)

3. SIGNATURE (Assinatura)

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret_key
)
```

- Garante que o token **não foi alterado**
- Apenas quem tem a **chave secreta** pode criar/validar

5.3 Claims (Reivindicações)

Claims Padrão (Registered Claims)

Claim	Nome	Descrição
iss	Issuer	Quem emitiu o token
sub	Subject	Sobre quem é o token (user ID)
aud	Audience	Para quem é o token
exp	Expiration	Quando expira (timestamp)
nbf	Not Before	Não válido antes de (timestamp)
iat	Issued At	Quando foi emitido (timestamp)
jti	JWT ID	Identificador único do token

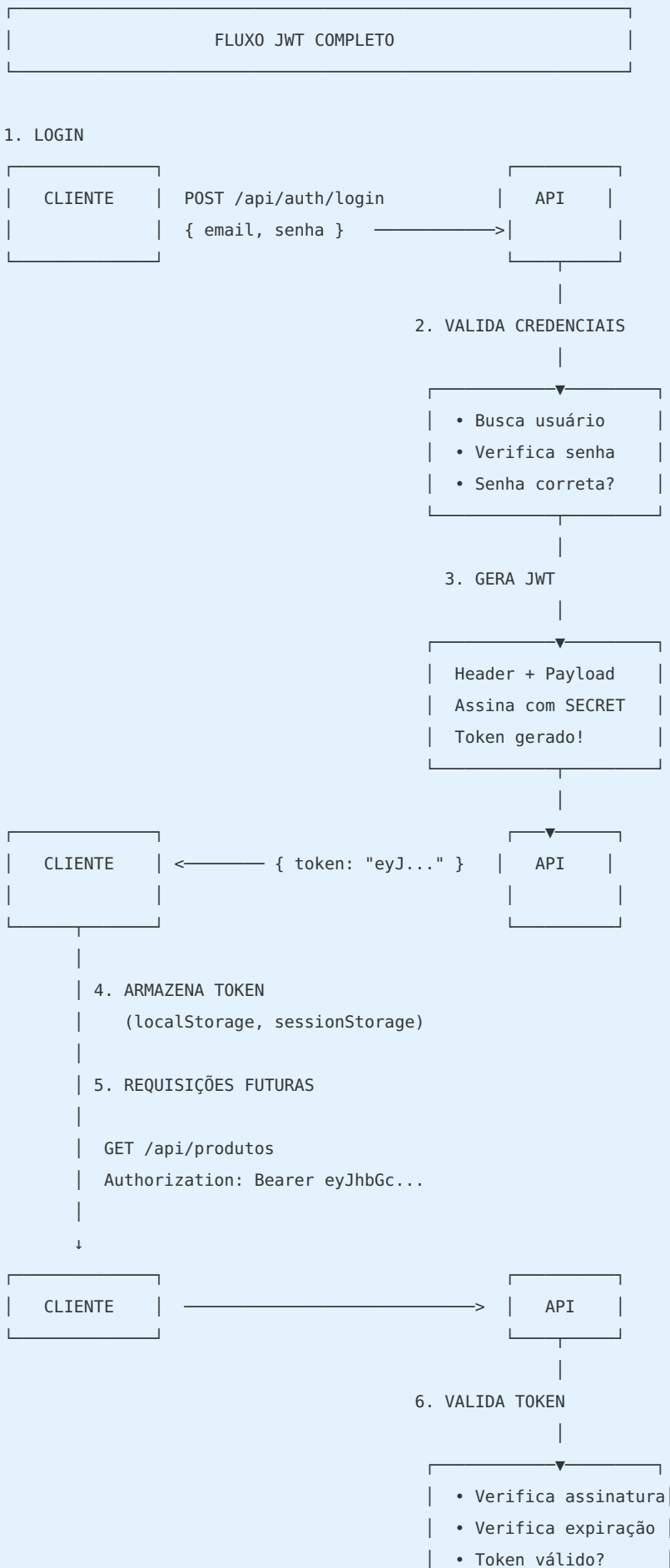
Claims Customizados

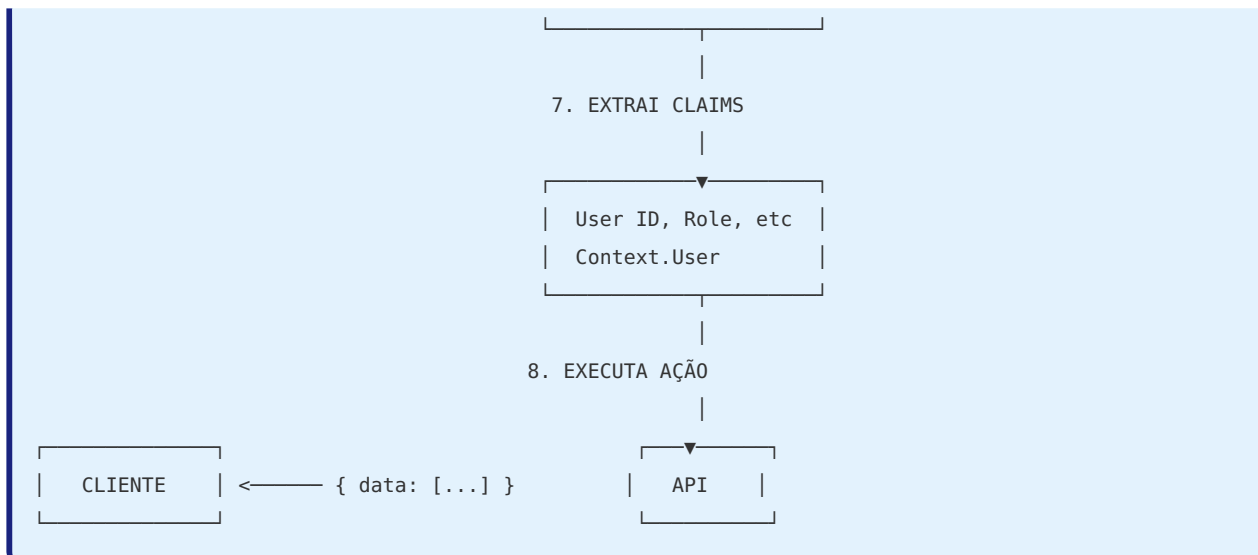
Você pode adicionar qualquer informação:

```
{
  "sub": "123",
  "email": "joao@email.com",
  "role": "Admin",
  "departamento": "TI",
  "permissoes": ["read", "write", "delete"]
}
```

 **ATENÇÃO:** Não coloque dados sensíveis (senhas, CPF, cartão)! JWT é **codificado**, não **criptografado** - qualquer um pode decodificar e ler!

5.4 Como JWT Funciona?





5.5 Vantagens do JWT

- ✓ **Stateless** - Servidor não armazena tokens
- ✓ **Escalável** - Funciona com load balancers
- ✓ **Autocontido** - Toda informação no token
- ✓ **Compacto** - Menor que cookies
- ✓ **Cross-domain** - Funciona em diferentes domínios
- ✓ **Mobile-friendly** - Ideal para apps nativos

5.6 Desvantagens do JWT

- ✗ **Tamanho** - Maior que session ID
- ✗ **Revogação** - Difícil invalidar antes da expiração
- ✗ **Não criptografado** - Dados visíveis (apenas codificados)
- ✗ **Single point of failure** - Se secret vazar, todos tokens comprometidos

6. IMPLEMENTAÇÃO JWT EM ASP.NET CORE

6.1 Instalação de Pacotes

```
# Pacote para autenticação JWT
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

# Pacote para criar tokens JWT
dotnet add package System.IdentityModel.Tokens.Jwt
```

6.2 Configuração no appsettings.json

```
{
  "Jwt": {
    "Key": "MinhaSuperChaveSecretaComPeloMenos32Caracteres123!",
    "Issuer": "https://api.einstein.edu.br",
    "Audience": "https://app.einstein.edu.br",
    "ExpireMinutes": 60
  }
}
```

 **IMPORTANTE:** - `Key` deve ter **pelo menos 32 caracteres** - **NUNCA** comitar a chave no Git - Usar **variáveis de ambiente** em produção

6.3 Configuração no Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// 1. Adicionar serviços
builder.Services.AddControllers();

// 2. Configurar autenticação JWT
builder.Services
    .AddAuthentication(options =>
    {
        options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
        options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    })
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            ),

            ClockSkew = TimeSpan.Zero // Remove margem de 5min padrão
        };
    });

// 3. Adicionar autorização
builder.Services.AddAuthorization();

var app = builder.Build();

// 4. Usar autenticação (ANTES de UseAuthorization!)
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

6.4 Criando o Serviço de Token

```

// Services/TokenService.cs
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

public interface ITokenService
{
    string GerarToken(Usuario usuario);
}

public class TokenService : ITokenService
{
    private readonly IConfiguration _configuration;

    public TokenService(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public string GerarToken(Usuario usuario)
    {
        // 1. Criar claims (informações do usuário)
        var claims = new[]
        {
            new Claim(JwtRegisteredClaimNames.Sub, usuario.Id.ToString()),
            new Claim(JwtRegisteredClaimNames.Email, usuario.Email),
            new Claim(JwtRegisteredClaimNames.Name, usuario.Nome),
            new Claim(ClaimTypes.Role, usuario.Role), // Admin, User, etc
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        // 2. Criar chave de assinatura
        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_configuration["Jwt:Key"])
        );

        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        // 3. Criar token
        var token = new JwtSecurityToken(
            issuer: _configuration["Jwt:Issuer"],
            audience: _configuration["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(
                Convert.ToDouble(_configuration["Jwt:ExpireMinutes"])
            ),
            signingCredentials: credentials
        );
    }
}

```



```
// 4. Retornar token como string
return new JwtSecurityTokenHandler().WriteToken(token);
}
}
```

Registrar no Program.cs:

```
builder.Services.AddScoped<ITokenService, TokenService>();
```

6.5 Controller de Autenticação

```

// Controllers/AuthController.cs
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly ITokenService _tokenService;
    private readonly IUsuarioRepository _usuarioRepository;

    public AuthController(
        ITokenService tokenService,
        IUsuarioRepository usuarioRepository)
    {
        _tokenService = tokenService;
        _usuarioRepository = usuarioRepository;
    }

    [HttpPost("login")]
    public async Task<ActionResult<LoginResponseDTO>> Login(LoginDTO dto)
    {
        // 1. Buscar usuário por email
        var usuario = await _usuarioRepository.GetByEmailAsync(dto.Email);

        if (usuario == null)
            return Unauthorized(new { message = "Credenciais inválidas" });

        // 2. Verificar senha (usar BCrypt ou similar)
        bool senhaCorreta = BCrypt.Net.BCrypt.Verify(dto.Senha, usuario.SenhaHash);

        if (!senhaCorreta)
            return Unauthorized(new { message = "Credenciais inválidas" });

        // 3. Gerar token JWT
        var token = _tokenService.GerarToken(usuario);

        // 4. Retornar token
        return Ok(new LoginResponseDTO
        {
            Token = token,
            Usuario = new UsuarioDTO
            {
                Id = usuario.Id,
                Nome = usuario.Nome,
                Email = usuario.Email,
                Role = usuario.Role
            }
        });
    }
}

```

```

[HttpPost("register")]
public async Task<ActionResult<LoginResponseDTO>> Register(RegisterDTO dto)
{
    // 1. Verificar se email já existe
    if (await _usuarioRepository.EmailExistsAsync(dto.Email))
        return BadRequest(new { message = "Email já cadastrado" });

    // 2. Criar usuário
    var usuario = new Usuario
    {
        Nome = dto.Nome,
        Email = dto.Email,
        SenhaHash = BCrypt.Net.BCrypt.HashPassword(dto.Senha),
        Role = "User" // Padrão
    };

    await _usuarioRepository.AddAsync(usuario);

    // 3. Gerar token
    var token = _tokenService.GerarToken(usuario);

    // 4. Retornar token
    return Ok(new LoginResponseDTO
    {
        Token = token,
        Usuario = new UsuarioDTO
        {
            Id = usuario.Id,
            Nome = usuario.Nome,
            Email = usuario.Email,
            Role = usuario.Role
        }
    });
}
}

```

6.6 DTOs

```
// DTOs/LoginDTO.cs
public class LoginDTO
{
    [Required(ErrorMessage = "Email é obrigatório")]
    [EmailAddress(ErrorMessage = "Email inválido")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Senha é obrigatória")]
    [MinLength(6, ErrorMessage = "Senha deve ter no mínimo 6 caracteres")]
    public string Senha { get; set; }
}

// DTOs/RegisterDTO.cs
public class RegisterDTO
{
    [Required]
    [MaxLength(100)]
    public string Nome { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [MinLength(6)]
    public string Senha { get; set; }
}

// DTOs/LoginResponseDTO.cs
public class LoginResponseDTO
{
    public string Token { get; set; }
    public UsuarioDTO Usuario { get; set; }
}

// DTOs/UsuarioDTO.cs
public class UsuarioDTO
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public string Email { get; set; }
    public string Role { get; set; }
}
```

6.7 Protegendo Endpoints

Exigir Autenticação

```
[Authorize] // ✅ Requer token válido
[HttpGet]
public ActionResult<List<ProdutoDTO>> GetAll()
{
    var produtos = _repository.GetAll();
    return Ok(produtos);
}
```

Exigir Role Específico

```
[Authorize(Roles = "Admin")] // ✅ Apenas Admin
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    _repository.Delete(id);
    return NoContent();
}
```

Múltiplos Roles

```
[Authorize(Roles = "Admin,Gerente")] // Admin OU Gerente
[HttpPut("{id}")]
public IActionResult Update(int id, ProdutoDTO dto)
{
    // ...
}
```

Endpoint Público (sem proteção)

```
[AllowAnonymous] // ✅ Não requer autenticação
[HttpGet("public")]
public ActionResult<List<ProdutoDTO>> GetPublic()
{
    // ...
}
```

6.8 Acessando Informações do Usuário

```
[Authorize]
[HttpGet("meu-perfil")]
public ActionResult<UsuarioDTO> GetMeuPerfil()
{
    // Extrair user ID do token
    var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var userId = int.Parse(userIdClaim);

    // Buscar usuário
    var usuario = _repository.GetById(userId);

    return Ok(new UsuarioDTO
    {
        Id = usuario.Id,
        Nome = usuario.Nome,
        Email = usuario.Email
    });
}
```

Helper para pegar User ID

```
public static class UserExtensions
{
    public static int GetUserId(this ClaimsPrincipal user)
    {
        var claim = user.FindFirst(ClaimTypes.NameIdentifier);
        return int.Parse(claim.Value);
    }

    public static string GetUserEmail(this ClaimsPrincipal user)
    {
        return user.FindFirst(ClaimTypes.Email)?.Value;
    }

    public static string GetUserRole(this ClaimsPrincipal user)
    {
        return user.FindFirst(ClaimTypes.Role)?.Value;
    }
}

// Uso:
[Authorize]
[HttpGet("perfil")]
public ActionResult GetPerfil()
{
    var userId = User.GetUserId();
    var email = User.GetUserEmail();
    var role = User.GetUserRole();

    // ...
}
```

7. BOAS PRÁTICAS DE SEGURANÇA

7.1 Senhas

✗ NUNCA armazene senhas em texto puro

```
✗ ERRADO:
usuario.Senha = "senha123"; // NUNCA FAÇA ISSO!
```


✓ SEMPRE use hash (BCrypt, Argon2)

✓ CORRETO:

```
using BCrypt.Net;

// Ao criar usuário
usuario.SenhaHash = BCrypt.HashPassword(dto.Senha);

// Ao verificar login
bool senhaValida = BCrypt.Verify(senhaDigitada, usuario.SenhaHash);
```

Instalar BCrypt:

```
dotnet add package BCrypt.Net-Next
```

7.2 Chave Secreta (Secret Key)

✗ NUNCA comitar no Git

✗ ERRADO (appsettings.json):

```
{
  "Jwt": {
    "Key": "minhasenha123" // ✗ Visível no repositório!
  }
}
```

✓ Usar variáveis de ambiente

Desenvolvimento (appsettings.Development.json):

```
{
  "Jwt": {
    "Key": "ChaveDeDesenvolvimentoNaoUsarEmProducao123!"
  }
}
```

Produção (variável de ambiente):

```
export JWT__KEY="ChaveSuperSecretaDeProd..."
```

Azure/AWS: - Usar **Azure Key Vault** - Usar **AWS Secrets Manager**

7.3 Expiração de Tokens

✓ Tokens devem expirar!

```
expires: DateTime.UtcNow.AddMinutes(60) // 1 hora
```

Tempos recomendados: - **Access Token:** 15 minutos a 1 hora - **Refresh Token:** 7 a 30 dias

7.4 HTTPS Obrigatório

```
// Program.cs
if (!app.Environment.IsDevelopment())
{
    app.UseHttpsRedirection(); // Força HTTPS
    app.UseHsts(); // HTTP Strict Transport Security
}
```

7.5 CORS Seguro

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("Production", policy =>
    {
        policy.WithOrigins("https://app.einstein.edu.br") // ✅ Específico
            .AllowAnyMethod()
            .AllowAnyHeader()
            .AllowCredentials();
    });
});

// ❌ NUNCA em produção:
// .AllowAnyOrigin() // Perigoso!
```

7.6 Rate Limiting

Limitar requisições para evitar abuso:

```
// Instalar: AspNetCoreRateLimit
dotnet add package AspNetCoreRateLimit

// Configurar no Program.cs
builder.Services.AddMemoryCache();
builder.Services.Configure<IpRateLimitOptions>(options =>
{
    options.GeneralRules = new List<RateLimitRule>
    {
        new RateLimitRule
        {
            Endpoint = "POST:/api/auth/login",
            Limit = 5, // 5 tentativas
            Period = "1m" // por minuto
        }
    };
});
```

7.7 Logging de Segurança

```
[HttpPost("login")]
public async Task<ActionResult> Login(LoginDTO dto)
{
    var usuario = await _repository.GetByEmailAsync(dto.Email);

    if (usuario == null)
    {
        _logger.LogWarning("Tentativa de login com email inexistente: {Email}", dto.Email);
        return Unauthorized();
    }

    if (!BCrypt.Verify(dto.Senha, usuario.SenhaHash))
    {
        _logger.LogWarning("Senha incorreta para usuário: {UserId}", usuario.Id);
        return Unauthorized();
    }

    _logger.LogInformation("Login bem-sucedido: {UserId}", usuario.Id);
    // ...
}
```

7.8 Validação de Entrada

```
[HttpPost]
public ActionResult Create([FromBody] ProdutoDTO dto)
{
    // 1. Validação automática (Data Annotations)
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    // 2. Validação de negócio
    if (dto.Preco < 0)
        return BadRequest("Preço não pode ser negativo");

    // 3. Sanitização
    dto.Nome = dto.Nome.Trim();

    // ...
}
```

8. REFRESH TOKENS

8.1 Problema: Tokens de Curta Duração

Access Token de 15 minutos expira rápido → usuário precisa fazer login novamente a cada 15 min! ❌

Solução: Refresh Token

8.2 Como Funciona

```
1. LOGIN
  ↓
Gera 2 tokens:
  • Access Token (15 min) - para acessar API
  • Refresh Token (7 dias) - para renovar access token
  ↓
2. USAR API
  Authorization: Bearer ACCESS_TOKEN
  ↓
3. ACCESS TOKEN EXPIRA
  ↓
4. RENOVAR TOKEN
  POST /api/auth/refresh
  { refreshToken: "..."}
  ↓
Retorna novo Access Token
  ↓
5. CONTINUAR USANDO API
```

8.3 Implementação

Modelo RefreshToken

```
public class RefreshToken
{
    public int Id { get; set; }
    public string Token { get; set; }
    public int UsuarioId { get; set; }
    public Usuario Usuario { get; set; }
    public DateTime Expiracao { get; set; }
    public bool Revogado { get; set; }
    public DateTime CriadoEm { get; set; }
}
```

Gerar Refresh Token

```
public class TokenService : ITokenService
{
    // ... GerarToken (access token)

    public RefreshToken GerarRefreshToken(int usuarioId)
    {
        return new RefreshToken
        {
            Token = Convert.ToBase64String(RandomNumberGenerator.GetBytes(64)),
            UsuarioId = usuarioId,
            Expiracao = DateTime.UtcNow.AddDays(7),
            Revogado = false,
            CriadoEm = DateTime.UtcNow
        };
    }
}
```

Endpoint de Login (com refresh)

```
[HttpPost("login")]
public async Task<ActionResult<LoginResponseDTO>> Login(LoginDTO dto)
{
    // ... validação de credenciais

    // Gerar access token
    var accessToken = _tokenService.GerarToken(usuario);

    // Gerar refresh token
    var refreshToken = _tokenService.GerarRefreshToken(usuario.Id);

    // Salvar refresh token no banco
    await _refreshTokenRepository.AddAsync(refreshToken);

    return Ok(new LoginResponseDTO
    {
        AccessToken = accessToken,
        RefreshToken = refreshToken.Token,
        ExpiresIn = 900 // 15 minutos em segundos
    });
}
```

Endpoint de Refresh

```
[HttpPost("refresh")]
public async Task<ActionResult<LoginResponseDTO>> Refresh(RefreshDTO dto)
{
    // 1. Buscar refresh token
    var refreshToken = await _refreshTokenRepository
        .GetByTokenAsync(dto.RefreshToken);

    if (refreshToken == null || refreshToken.Revogado)
        return Unauthorized(new { message = "Token inválido" });

    // 2. Verificar expiração
    if (refreshToken.Expiracao < DateTime.UtcNow)
        return Unauthorized(new { message = "Token expirado" });

    // 3. Buscar usuário
    var usuario = await _usuarioRepository.GetByIdAsync(refreshToken.UsuarioId);

    // 4. Gerar novo access token
    var novoAccessToken = _tokenService.GerarToken(usuario);

    // 5. Opcionalmente, revogar refresh token antigo e gerar novo
    refreshToken.Revogado = true;
    await _refreshTokenRepository.UpdateAsync(refreshToken);

    var novoRefreshToken = _tokenService.GerarRefreshToken(usuario.Id);
    await _refreshTokenRepository.AddAsync(novoRefreshToken);

    return Ok(new LoginResponseDTO
    {
        AccessToken = novoAccessToken,
        RefreshToken = novoRefreshToken.Token,
        ExpiresIn = 900
    });
}
```

9. HTTPS E OUTRAS CAMADAS DE SEGURANÇA

9.1 HTTPS (SSL/TLS)

Por quê? -  Criptografa dados em trânsito -  Previne Man-in-the-Middle -  Previne eavesdropping (espionagem) -  Confiança (cadeado verde)

Como habilitar:

```
// Program.cs
app.UseHttpsRedirection(); // Redireciona HTTP → HTTPS

// Em produção, usar certificado SSL
// - Let's Encrypt (gratuito)
// - Certificado pago
// - Azure/AWS gerenciam automaticamente
```

9.2 HSTS (HTTP Strict Transport Security)

```
app.UseHsts(); // Força HTTPS por período definido

builder.Services.AddHsts(options =>
{
    options.MaxAge = TimeSpan.FromDays(365);
    options.IncludeSubDomains = true;
});
```

9.3 Security Headers

```
app.Use(async (context, next) =>
{
    // Previne clickjacking
    context.Response.Headers.Add("X-Frame-Options", "DENY");

    // Previne MIME sniffing
    context.Response.Headers.Add("X-Content-Type-Options", "nosniff");

    // XSS Protection
    context.Response.Headers.Add("X-XSS-Protection", "1; mode=block");

    // Content Security Policy
    context.Response.Headers.Add("Content-Security-Policy",
        "default-src 'self'; script-src 'self'");

    await next();
});
```


9.4 Validação de Origem (CORS)

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("Producao", policy =>
    {
        policy.WithOrigins(
            "https://app.einstein.edu.br",
            "https://mobile.einstein.edu.br"
        )
        .AllowAnyMethod()
        .AllowAnyHeader()
        .AllowCredentials();
    });
});

app.UseCors("Producao");
```

10. CHECKLIST DE SEGURANÇA

10.1 Autenticação

- [] JWT implementado corretamente
- [] Tokens têm expiração (15min a 1h)
- [] Refresh tokens implementados
- [] Senhas com hash (BCrypt, Argon2)
- [] Nunca armazenar senhas em texto puro
- [] Validação de email e senha no login
- [] Rate limiting em endpoint de login (máx 5 tentativas/min)

10.2 Autorização

- [] `[Authorize]` em endpoints protegidos
- [] Roles implementados corretamente
- [] Verificação de permissões no código
- [] BOLA prevenido (usuário só acessa seus recursos)
- [] Endpoints admin só para `[Authorize(Roles = "Admin")]`

10.3 Configuração

- [] HTTPS em produção
- [] Chave JWT secreta (32+ caracteres)
- [] Chave JWT em variável de ambiente (não no código)

- [] CORS configurado corretamente (sem `AllowAnyOrigin`)
- [] Security headers configurados

10.4 Dados

- [] Não expor dados sensíveis em responses (usar DTOs)
- [] Validação de entrada em todos endpoints
- [] Sanitização de strings
- [] SQL Injection prevenido (usar EF Core, não SQL raw)
- [] XSS prevenido (validação de entrada)

10.5 Logging e Monitoramento

- [] Logar tentativas de login (sucesso e falha)
- [] Logar acessos não autorizados
- [] Monitorar tentativas de brute force
- [] Alertas para atividades suspeitas

10.6 Testes

- [] Testar login com credenciais válidas
- [] Testar login com credenciais inválidas
- [] Testar acesso sem token (deve retornar 401)
- [] Testar acesso com token expirado (deve retornar 401)
- [] Testar acesso a endpoint admin sem role (deve retornar 403)
- [] Testar refresh token

RESUMO - PONTOS-CHAVE

JWT em 5 Passos

1. **Login** → usuário envia email + senha
2. **Validar** → API verifica credenciais
3. **Gerar JWT** → API cria token assinado com claims
4. **Retornar** → API retorna token para cliente
5. **Usar** → Cliente envia token em `Authorization: Bearer TOKEN`

Estrutura JWT

HEADER . PAYLOAD . SIGNATURE

- **Header:** Algoritmo (HS256, RS256)

- **Payload:** Claims (userId, role, email, exp)
- **Signature:** HMACSHA256(header + payload, secret)

Código Essencial

```
// 1. Configurar JWT (Program.cs)
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => { /* ... */ });

// 2. Gerar token (TokenService)
var token = new JwtSecurityToken(
    issuer: "...",
    audience: "...",
    claims: claims,
    expires: DateTime.UtcNow.AddMinutes(60),
    signingCredentials: credentials
);

// 3. Proteger endpoint (Controller)
[Authorize]
[HttpGet]
public ActionResult Get() { }

[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult Delete(int id) { }
```

Boas Práticas

- ✓ HTTPS obrigatório
- ✓ Tokens com expiração curta (15-60 min)
- ✓ Refresh tokens para renovação
- ✓ Senhas com BCrypt
- ✓ Chave secreta em variável de ambiente
- ✓ Rate limiting em login
- ✓ Logging de tentativas de acesso
- ✓ CORS configurado (sem AllowAnyOrigin)
- ✓ Validação de entrada
- ✓ Usar DTOs (não expor dados sensíveis)

REFERÊNCIAS

Documentação Oficial

- **JWT:** <https://jwt.io/>
- **Microsoft JWT:** <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/>

- **OWASP API Security:** <https://owasp.org/www-project-api-security/>

Ferramentas

- **JWT Debugger:** <https://jwt.io/>
- **BCrypt Online:** <https://bcrypt-generator.com/>
- **Postman:** <https://www.postman.com/>

Leitura Complementar

- **OAuth 2.0:** <https://oauth.net/2/>
- **OpenID Connect:** <https://openid.net/connect/>
- **Security Best Practices:** <https://cheatsheetseries.owasp.org/>

Elaborado por: Prof. Geovani

Instituição: Faculdade Einstein de Limeira

Disciplina: Desenvolvimento Web Back-End

Versão: 1.0 - 2026